

Block Drill Exam — Set 4

Twelve blocks across new domains: strings, dates, regex, classes, generics, streams, auth; answers follow each block

October 2026

How to run it.

Same method. 8 minutes for 1–6, 12 for 7–12. Three of the twelve are clean.

Block 1 — Slugify

Requirement: turn a title into a URL slug: lowercase, spaces and runs of non-alphanumerics to single `-`, trimmed of leading/trailing `-`; empty input yields `""`.

```
1 function slugify(title: string): string {
2   title.toLowerCase();
3   const s = title.replace(/^[a-z0-9]+/g, "-");
4   return s.replace(/^-|-$/g, "");
5 }
```

Notes

Line	Note
2	result dropped; <code>title</code> unchanged (strings immutable)
3	runs on the ORIGINAL case; uppercase letters are "non-alphanumeric" to <code>[^a-z0-9]</code> and become <code>-</code> ; <code>"Hello World"</code> → <code>"-ello-world"</code>
4	<code>/^- -\$/</code> without <code>g</code> removes only the FIRST match: a leading <code>-</code> is removed, a trailing one is not (or vice versa); <code>"-a-"</code> → <code>"a-"</code>

Findings

- Line 2: `toLowerCase()` result discarded; `const t = title.toLowerCase()` and use `t` on line 3.
- Line 4: regex needs the `g` flag to strip both ends: `/^-+|-+$/g`.
- Line 3: with the fix on line 2 this is correct; without it, every capital letter is replaced.

Confidence 1 · Reject.

Wrong on any title with a capital letter.

Block 2 — Days between

Requirement:

whole days between two ISO date strings, non-negative; throw on an invalid date.

```
1 function daysBetween(a: string, b: string): number {
2   const da = new Date(a), db = new Date(b);
3   if (!da || !db) throw new Error("invalid date");
4   return Math.abs(db - da) / 86_400_000;
5 }
```

Notes

Line	Note
2	<code>new Date("garbage")</code> is an Invalid Date OBJECT, truthy
3	<code>!da</code> is never true; invalid dates pass
4	<code>db - da</code> on Date objects coerces to numbers (ms): works at runtime; TS flags arithmetic on Dates as a type error; <code>NaN</code> for an invalid date; <code>/ 86_400_000</code> gives fractional days, not whole; DST transitions make some days 23 or 25 h, so "whole days" via division is off by a fraction

Findings

- 1. Line 3: check `Number.isNaN(da.getTime())`.
- 2. Line 4: `Math.floor` (or `Math.round`) for whole days; use `.getTime()` explicitly for the type checker.
- 3. Line 4: DST: for calendar days, compare UTC dates or `Date.UTC(y, m, d)` of each; for elapsed 24 h periods, the division is fine; the requirement says “whole days”, so note the ambiguity.

Confidence 2 · Request changes.

Block 3 — Mask card number

Requirement: return the card number with all but the last 4 digits replaced by *, preserving spaces; input may contain spaces.

```
1 function mask(card: string): string {
2   const digits = card.replace(/\s/g, "");
3   const visible = digits.slice(-4);
4   return "*".repeat(digits.length - 4) + visible;
5 }
```

Notes

Line	Note
2	spaces removed: the "preserving spaces" clause is dropped
3	last 4
4	digits.length - 4 is negative for inputs shorter than 4; "".repeat(-1) throws RangeError

Findings

- 1. Line 2/4: spaces are not preserved; walk the original string and replace digit characters only, keeping the last 4 digits.
- 2. Line 4: inputs shorter than 4 characters throw; clamp with Math.max(0, ...).

Confidence 2 · Request changes.

Block 4 — Class with getter cache

Requirement: `Report.total` is the sum of `items[].amount`; it must reflect items added after construction.

```
1 class Report {
2   private _total?: number;
3   constructor(public items: Item[]) {}
4   get total(): number {
5     if (this._total === undefined) this._total = this.items.reduce((s, i) => s + i.amount, 0);
6     return this._total;
7   }
8   add(item: Item) { this.items.push(item); }
9 }
```

Notes

Line	Note
2	memo slot
3	<code>items</code> is public and is the caller's array (alias)
5	computes once, then cached forever; <code>=== undefined</code> check is correct (a total of 0 would be cached and not recomputed, which is intended)
8	pushes but does not clear <code>_total</code> : the cached total is stale after <code>add</code>
3	the caller can also <code>items.push</code> directly, bypassing <code>add</code>

Findings

- Line 8: `add` must invalidate: `this._total = undefined`.
- Line 3: `public items` exposes the array; any external push leaves the cache stale; make it private or recompute every time (the sum is cheap).

Confidence 2 · Request changes.

The one clause (reflect later adds) fails.

Block 5 — Pick fields

Requirement: `pick(obj, keys)` returns a new object with only the given keys; missing keys are omitted (not set to `undefined`).

```
1 function pick<T extends object, K extends keyof T>(obj: T, keys: K[]): Pick<T, K> {
2   const out = {} as Pick<T, K>;
3   for (const k of keys) {
4     if (k in obj) out[k] = obj[k];
5   }
6   return out;
7 }
```

Notes

Line	Note
1	generic constrained to the object's keys: callers cannot pass a bad key
2	<code>{}</code> as <code>Pick<T, K></code> is a cast on an empty object; acceptable in a builder that fills it; the type is honest only if all keys exist, which line 4 does not guarantee: the return type claims required fields that may be omitted (type-level nit)
4	<code>k in obj</code> includes inherited keys; for plain data objects this is fine; a key present with value <code>undefined</code> IS copied (requirement says missing keys omitted; a present-but-undefined key is not "missing")
6	new object; values shared (shallow)

Findings

- Nit: return type should be `Partial<Pick<T, K>>` since keys can be omitted.
- Nit: `in` vs `Object.hasOwn`; matters only for prototype keys.

Confidence 4 · Approve.

Behavior matches; the type is slightly optimistic.

Block 6 — Parallel with limit (generic)

Requirement: `mapLimit(items, limit, fn)` maps with at most `limit` concurrent `fn` calls; results in input order; first error rejects.

```
1  async function mapLimit<T, R>(items: T[], limit: number, fn: (t: T) => Promise<R>):  
Promise<R[]> {  
2    const results: R[] = new Array(items.length);  
3    let i = 0;  
4    const worker = async () => {  
5      while (i < items.length) {  
6        const idx = i++;  
7        results[idx] = await fn(items[idx]);  
8      }  
9    };  
10   await Promise.all(Array.from({ length: Math.min(limit, items.length) }, worker));  
11   return results;  
12 }
```

Notes

Line	Note
3	shared counter among workers
5–6	read-then-increment with NO await between: <code>i++</code> is atomic in JS; each worker claims a unique index
7	awaits, writes by index: order preserved
10	<code>Math.min(limit, items.length)</code> workers; <code>limit <= 0</code> → zero workers → resolves <code>[]</code> -length array of holes (edge); a rejection in any worker rejects <code>Promise.all</code> , other workers keep running (acceptable: "first error rejects")

Findings

- Edge: validate `limit >= 1`.

Confidence 5 · Approve. The shared `i` looks like a race and is not: there is no `await` between read and increment.

Block 7 — Auth middleware

Requirement: verify a Bearer JWT; attach `req.user`; 401 if missing/invalid/expired; never leak why.

```
1 function auth(req: Request, res: Response, next: NextFunction) {
2   const [scheme, token] = (req.headers.authorization ?? "").split(" ");
3   if (scheme !== "Bearer") return res.status(401).json({ error: "no bearer" });
4   try {
5     const payload = jwt.verify(token, SECRET) as Claims;
6     if (payload.exp < Date.now()) return res.status(401).json({ error: "expired" });
7     req.user = payload;
8     next();
9   } catch (e) {
10    res.status(401).json({ error: (e as Error).message });
11  }
12 }
```

Notes

Line	Note
2	<code>?? ""</code> then split: <code>scheme</code> is <code>""</code> and <code>token</code> is <code>undefined</code> when absent; fine
3	returns; message leaks the reason (clause: never leak why)
5	<code>jwt.verify</code> is sync here (callback-less form); it already checks <code>exp</code> and throws on expiry; <code>as Claims</code> unverified shape
6	<code>payload.exp</code> is in SECONDS (JWT standard); <code>Date.now()</code> is MILLISECONDS; the comparison is wrong by 1000×: every token looks expired... no: <code>exp</code> (seconds, ~1.7e9) < <code>Date.now()</code> (ms, ~1.7e12) is ALWAYS true, so every valid token is rejected as expired; except <code>jwt.verify</code> already threw for truly expired ones
10	leaks the library's error message (clause violated)
8	<code>next()</code> inside <code>try</code> : an error thrown by a downstream handler synchronously is caught here and reported as 401 (subtle)

Findings

- 1. Line 6: units mismatch (`exp` seconds vs `Date.now()` ms); every token is rejected; remove the check (`jwt.verify` does it) or compare to `Date.now() / 1000`.
- 2. Lines 3, 10: error messages leak the reason; respond with a constant body.
- 3. Line 8: `next()` inside the `try` catches downstream errors; move it after.
- 4. Line 5: `as Claims` unverified; validate the fields you use.

Confidence 1 • Reject.

No valid request passes.

Block 8 — Stream to string

Requirement:

collect a readable stream into a string; reject on stream error; cap at 10 MB with a specific error.

```
1 function readAll(stream: Readable, max = 10 * 1024 * 1024): Promise<string> {
2   return new Promise((resolve, reject) => {
3     const chunks: Buffer[] = [];
4     let size = 0;
5     stream.on("data", (c: Buffer) => {
6       size += c.length;
7       if (size > max) reject(new Error("too large"));
8       chunks.push(c);
9     });
10    stream.on("end", () => resolve(Buffer.concat(chunks).toString("utf8")));
11    stream.on("error", reject);
12  });
13 }
```

Notes

Line	Note
7	rejects but does NOT stop the stream: <code>data</code> events keep arriving, chunks keep accumulating (memory keeps growing past the cap), and <code>end</code> later calls <code>resolve</code> on an already-rejected Promise (ignored, harmless)
8	pushes even the over-limit chunk
10	<code>Buffer.concat</code> then decode: correct for multi-byte UTF-8 across chunk boundaries (decoding per chunk would split characters)
11	error wired

Findings

- 1. Line 7: after rejecting, `stream.destroy()` (or `stream.removeAllListeners` and `unpipe`) so no more data accumulates; otherwise the cap does not cap memory.
- 2. Line 8: `return` after the reject so the over-limit chunk is not pushed (minor once 1 is fixed).

Positives: concat-then-decode handles multi-byte boundaries; `error` listener present.

Confidence 3 • Request changes.

Block 9 — Enum mapping

Requirement: map a numeric status code to a label; unknown codes return "unknown".

```
1  enum Status { Active, Inactive, Banned }
2  const LABEL: Record<Status, string> = { [Status.Active]: "active", [Status.Inactive]:
"inactive", [Status.Banned]: "banned" };
3  function label(code: number): string {
4    return LABEL[code as Status] ?? "unknown";
5  }
```

Notes

Line	Note
1	numeric enum: Active = 0, Inactive = 1, Banned = 2
2	Record<Status, string> forces all three keys at compile time: adding a fourth member is a compile error here (good)
4	code as Status is a cast for the index; LABEL[99] is undefined; ?? "unknown" handles it; LABEL[0] is "active" and 0 is falsy but ?? does not care: correct choice over

Findings

- None. (A || on line 4 would have been a bug; ?? is right.)

Confidence 5 · Approve.

Block 10 — Shallow equal

Requirement: `shallowEqual(a, b)` is true if both objects have the same keys with `===` values.

```
1 function shallowEqual(a: Record<string, unknown>, b: Record<string, unknown>): boolean {
2   for (const k in a) if (a[k] !== b[k]) return false;
3   return true;
4 }
```

Notes

Line	Note
2	checks every key of <code>a</code> against <code>b</code> ; a key present in <code>b</code> but absent in <code>a</code> is never checked: <code>shallowEqual({}, { x: 1 })</code> is <code>true</code>
2	<code>for...in</code> includes inherited enumerable keys (edge)
2	<code>a[k] !== b[k]</code> treats a missing key in <code>b</code> as <code>undefined</code> , so <code>{ x: undefined }</code> equals <code>{}</code> (edge)
2	<code>NaN !== NaN</code> : objects with <code>NaN</code> values never equal (edge; <code>Object.is</code>)

Findings

- 1. Line 2–3: asymmetric; compare key counts first (`Object.keys(a).length !== Object.keys(b).length` → false), then iterate.
- 2. Edge: `Object.is` for `NaN`; `Object.hasOwn` for key presence.

Confidence 2 · Request changes.

Wrong on the stated requirement (“same keys”).

Block 11 — Retry on specific errors

Requirement: retry `op` only when it throws a `RetryableError`; up to 3 attempts; other errors propagate immediately.

```
1  async function retryIf<T>(op: () => Promise<T>): Promise<T> {
2    for (let i = 0; i < 3; i++) {
3      try {
4        return await op();
5      } catch (e) {
6        if (!(e instanceof RetryableError)) throw e;
7      }
8    }
9    throw new Error("retries exhausted");
10 }
```

Notes

Line	Note
2	three attempts
4	<code>return await</code> in <code>try</code> : caught
6	non-retryable rethrown immediately: clause satisfied; retryable falls through to the next iteration
9	after three retryable failures, a NEW generic error is thrown; the last <code>RetryableError</code> is lost (its message/cause)

Findings

1. Line 9: the original error is discarded; keep `let last: unknown` and `throw last` (or wrap with `{ cause: last }`).

Confidence 4 · Approve with one change.

Logic is correct; diagnostics are lost.

Block 12 — The clean one

Requirement: `groupInto(items, keyFn)` returns a `Map` from key to array of items, in first-seen key order; input untouched.

```
1 function groupInto<T, K>(items: readonly T[], keyFn: (t: T) => K): Map<K, T[]> {
2   const out = new Map<K, T[]>();
3   for (const it of items) {
4     const k = keyFn(it);
5     const list = out.get(k);
6     if (list) list.push(it); else out.set(k, [it]);
7   }
8   return out;
9 }
```

Notes

Line	Note
1	<code>readonly</code> input; generic key type so object keys compare by identity (document)
5–6	<code>get</code> then truthy check: <code>list</code> is an array or <code>undefined</code> ; arrays are truthy even when empty, and no empty array is ever stored, so the check is safe
6	pushes to the stored array (own) or creates a new one
8	<code>Map</code> preserves first-seen order

Findings

- None.

Confidence 5 · Approve.

Scoring yourself

Block	Verdict	Must-find
1	reject	dropped <code>toLowerCase</code> , regex without <code>g</code>
2	request changes	Invalid Date truthy, fractional days
3	request changes	spaces dropped, negative repeat
4	request changes	stale cache after <code>add</code>
5	approve	type nit only
6	approve	the shared <code>i</code> is not a race
7	reject	<code>exp</code> seconds vs ms
8	request changes	stream not destroyed after cap
9	approve	<code>??</code> is right
10	request changes	asymmetric
11	approve with one change	original error lost
12	approve	